Prospect Tracker (v1) — Local-First CRUD System

Audience	Technical reviewers
Scope	System design documentation
Prepared by	Jhondel Mellomida
Last updated	5/10/2026

1. Overview

1.1 Problem statement (why build vs. spreadsheet)

This tracking system is meant to track client acquisition and its progress, and whether it is worth continuing or not. It is meant to simplify the process and improve the setup for that task.

Rather than using a spreadsheet for tracking, it was decided to build a system to practice coding from scratch, with AI as a guide only and not as a code generator. It is a learning approach, which is why we are building it. Another advantage is the flexibility to shape it around what we want to achieve.

1.2 Goals / Non-goals

Goals

- Serve as a learning exercise for cache-aside invalidation patterns, not just a CRUD app.

Non-goals

- Fine-grained/partial cache invalidation (v1 clears the whole cache on any write).
- Distributed caching (Redis deferred to v2).

2. Requirements

2.1 Functional requirements

ID	Requirement	Priority
FR-1.1	User can create an application record with company, contact (email/number), date created, last date contacted, technical audit (free text), source, and status. Company and status are required.	Must
FR-1.2	User can edit an existing application record.	Must
FR-1.3	User can delete an existing application record.	Must
FR-1.4	User can see an individual application record.	Must
FR-1.5	User can page through records.	Must
FR-2.1	User can add notes to an application record, such as a client response. Notes are auto-timestamped.	Must
FR-2.2	User can edit notes on an application record, such as a client response.	Must
FR-2.3	User can delete notes on an application record, such as a client response.	Must

ID	Requirement	Priority
FR-3.1	User can filter based on status.	Must
FR-3.2	User can sort records by last contacted date, ascending or descending.	Must
FR-3.3	User can search by company and source (case-insensitive, partial match).	Must
FR-4.1	User can manually set a follow-up reminder date. Records due today or earlier are flagged in the list.	Should
FR-5.1	User can export records to CSV.	Could
FR-5.2	User can import records from CSV.	Could
FR-6.1	Login, accounts, and profile management.	Won't
FR-7.1	System writes an audit log entry (entity, action, old/new values as strings, timestamp) on every create, update, or delete of a record, note, or follow-up.	Must
FR-8.1	System passes telemetry data for monitoring and analysis.	Won't

2.2 Non-functional requirements

ID	Category	Requirement
NFR-1	Performance	List endpoint p95 < 500 ms at up to 1,000 records.
NFR-2	Performance	Search endpoint p95 < 500 ms upon clicking the search button.
NFR-3	Data Integrity	Deleting a record cascades to its notes and follow-up schedule; no orphans. Audit logs are excluded and survive deletion.
NFR-4	Reliability	Schema changes only via Django migrations (makemigrations/migrate); every migration is reversible.
NFR-5	Auditability	Audit entries are timestamped (UTC) and append-only (no edit or delete endpoint).
NFR-6	Security	API bound to localhost or a private network; v1 is single user with no auth.
NFR-7	Maintainability	Backend test coverage ≥ 80% on Django views/viewsets.
NFR-8	Maintainability	Code passes ruff (PEP 8) in CI; source files ≤ 200 lines unless justified in a comment.
NFR-9	Performance	GET /records list is cached in-process (dict, TTL 30 s) and invalidated on any write to records or notes. Django's LocMemCache will be used.
NFR-10	Performance	The list endpoint response time target (NFR-1) applies with and without a cache hit; the cache is an optimization, not a requirement for meeting the target.

3. System Interface Definition

REST was used as the main architectural style for its simplicity, since the project is mostly CRUD and monolithic. GraphQL is an alternative; however, changing the architectural style would mean changing the endpoints.

3.1 API endpoints

Records

Method	Path	Purpose	Satisfies	Success	Errors
POST	`/api/v1/records`	Add application record	FR-1.1	201	400
PATCH	`/api/v1/records/{record_id}`	Edit application record	FR-1.2, FR-4.1	200	404, 400
DELETE	`/api/v1/records/{record_id}`	Delete application record	FR-1.3	204	404
GET	`/api/v1/records/{record_id}`	Get one application record with its notes	FR-1.4	200	404
GET	`/api/v1/records`	List records (filter, sort, search, due)	FR-1.5, FR-3.1, FR-3.2, FR-3.3, FR-4.1	200	400
GET	`/api/v1/records/export`	Download all records as CSV	FR-5.1	200	None
POST	`/api/v1/records/import`	Upload CSV to records	FR-5.2	201	400

Notes

Method	Path	Purpose	Satisfies	Success	Errors
POST	`/api/v1/records/{record_id}/notes`	Add a note to a record	FR-2.1	201	404, 400
PATCH	`/api/v1/notes/{note_id}`	Edit a note	FR-2.2	200	404, 400
DELETE	`/api/v1/notes/{note_id}`	Delete a note	FR-2.3	204	404

Audit

Method	Path	Purpose	Satisfies	Success	Errors
GET	`/api/v1/audit-logs`	Read-only audit logs, filterable	FR-7.1	200	400

GET /api/v1/records: query parameters and response

Query parameters

Parameter	Required	Description
`status`	Optional	One of the allowed status values.
`q`	Optional	Case-insensitive partial match on company or source.
`sort_by`	Optional	Default: last_date_contacted.
`order`	Optional	asc | desc. Default: desc.
`follow_up_due`	Optional	true returns only records due today or earlier.
`page`	Optional	Default: 1.

Parameter	Required	Description
page_size	Optional	One of 10 | 20 | 50 | 100. Default: 20.

Response body

```
{
  "items": [ ...records... ],
  "page": 1,
  "page_size": 20,
  "total": 137,
  "total_pages": 7
}
```

Errors

400 if page_size is not one of the allowed values, or page < 1.

3.2 Request/response contract notes

Topic	Decision	Why
Versioning	/api/v1 prefix.	Project is still in v1 and to create a standard.
Naming	Plural nouns, snake_case in paths, params, and JSON.	Follows PEP 8 (NFR-8) and DRF / Django conventions.
IDs	Integer, auto-increment.	Single user and easier to set up for v1.
Dates	Date fields (last_date_contacted and follow_up_date) follow YYYY-MM-DD; created_at and updated_at as ISO 8601 UTC with Z.	A date field is not a moment in time, so it shouldn't carry a timezone.
Update semantics	PATCH only.	Users edit one field at a time, and PUT would force sending everything.
Success codes	POST 201 with body, GET 200, PATCH 200 with updated object, DELETE 204 with no body.	One rule per method. PATCH returns the object so the UI doesn't need a second GET.
Error shape	{"detail": "..."} for 404 and 400, and keep Django's default 400 body for validation.	Zero custom code, and the client handles one top-level key.
Error codes	Document only 404 and 400 (import). 405 and 500 are framework or infrastructure concerns, not per-endpoint design.	Every listed code should be something your handler deliberately produces.
Pagination	Page-number based (page, page_size), not cursor-based. Allowed sizes: 10, 20, 50, 100. Response wraps items in an envelope with page, page_size, total, total_pages.	Cursor pagination solves problems you don't have (high write concurrency, huge datasets). Page number is simpler to implement and to page through in a UI with numbered buttons.
Auth / CORS	None (NFR-6). CORS allows only the Next.js origin (e.g., http://localhost:3000).	Stated explicitly so it's a decision, not an omission.
Caching	Server-side, in-process dict, TTL 30 s, full invalidation on any record/note write. No HTTP cache headers (Cache-Control/ETag) in v1.	Keeps cache logic entirely server-side and simple to reason about; avoids conflating two different caching mechanisms.

4. Estimation

4.1 Scale assumptions (even at trivial scale)

Dimension	Assumption	Basis
Users	1	Single user v1
Records (total)	~540 over 6 months	3/day average (realistic range 1–5/day) × 180 days
Records (total, ceiling)	1,800 over 6 months	10/day stretch goal × 180 days
Records (growth rate)	~3/day (~90/month) steady state	Anchor metric
Notes per record	3	Initial, ongoing, and conclusion
Writes per record (lifetime)	~6	1 create + 2 edits + 3 notes
Requests per day	~38 (~18 writes + ~20 reads)	Derived from records/day × lifetime writes, and estimated session count
Read:write ratio	~1:1	Recomputed, not assumed
Peak concurrency	1	Single user v1
Data size (rough)	~972 KB (~0.95 MB) (~97.2 KB + ~291.6 KB + ~583.2 KB)	~180 bytes per row for records (~540), notes (~540 × 3 = ~1,620), and audit logs (~540 × 6 = ~3,240)

5. Data Model

5.1 Schema

```
Table records {
  record_id integer [pk, not null, increment]
  company string [not null]
  status status [not null]
  company_contact_no string
  company_email email
  last_date_contacted date
  technical_audit string [note: "Free form text for the audit report summary"]
  source source [note: "Application where the client data comes from"]
  created_at timestamp [not null, default: `now()`]
  updated_at timestamp
}

Table notes {
  note_id integer [pk, not null, increment]
  record_id integer [not null]
  content string
  created_at timestamp [not null, default: `now()`]
  updated_at timestamp
}

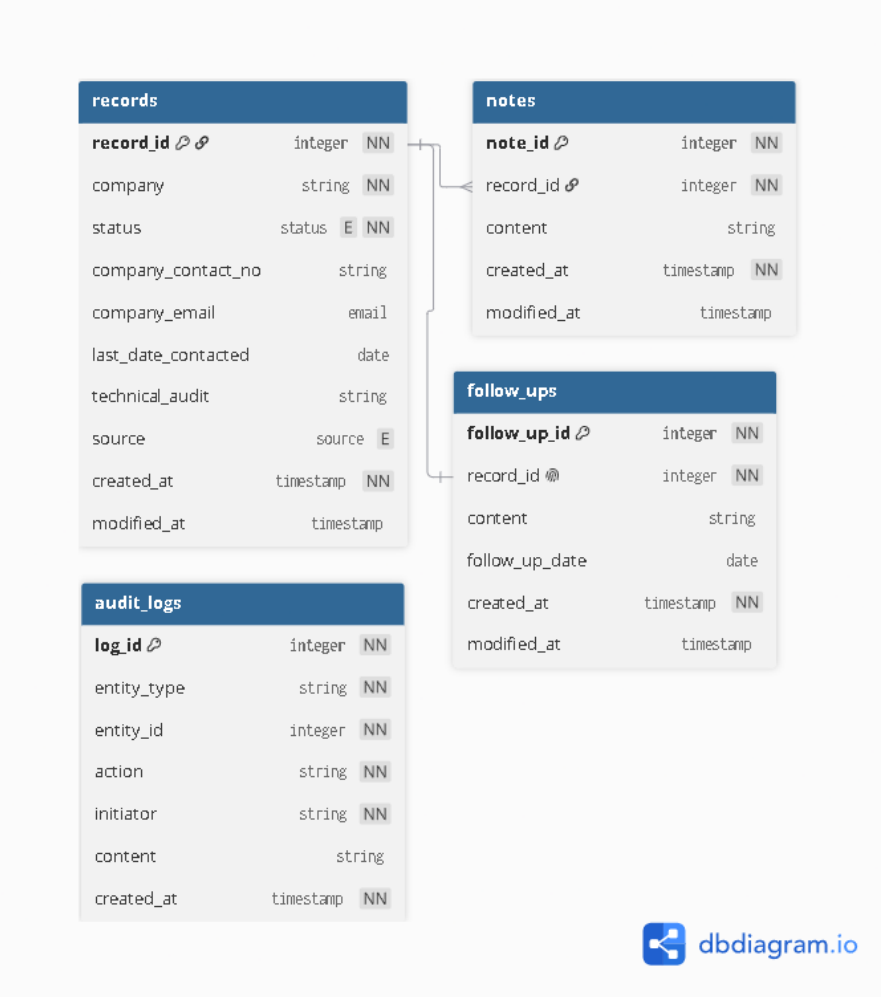
Table follow_ups {
  follow_up_id integer [pk, not null, increment]
  record_id integer [not null, unique]
  content string
  follow_up_date date
  created_at timestamp [not null, default: `now()`]
  updated_at timestamp
}

Table audit_logs {
  log_id integer [pk, not null, increment]
  entity_type string [not null, note: "'record' | 'note' | 'follow_up' — always explicit, no
default"]
  entity_id integer [not null, note: "No FK on purpose — must survive deletion of the parent entity
(NFR-3)"]
  action string [not null, note: "'create' | 'update' | 'delete'"]
  initiator string [not null, default: "system", note: "Placeholder until v2 auth adds real users"]
  old_value string [note: "null on create"]
  new_value string [note: "null on delete"]
  created_at timestamp [not null, default: `now()`]
}

Ref: notes.record_id > records.record_id [delete: cascade]
Ref: follow_ups.record_id - records.record_id [delete: cascade]

enum status {
  contacted
  ongoing
  completion
  dropped
}

enum source {
  reddit
  discord
  facebook
  linkedin
  others
}
```



5.2 Decision: fixed follow-up columns vs. separate follow_ups table

ADR-0001: Separate follow_ups table vs. fixed columns on records

Status: Accepted

Context

FR-4.1 describes one follow-up per record, so the relationship is one-to-one. This made fixed columns a legitimate option worth weighing seriously, not a table-by-default choice. The deciding question was whether that one-to-one assumption is likely to hold beyond v1.

Decision

Decided to use follow-ups as a separate table.

Alternatives considered

Option A: Fixed columns on records

- **Pros:** Simplicity, easy access from records without a join.
- **Cons:** No extra metadata and not normalized.

Option B: Separate follow_ups table

- **Pros:** Normalized, cleaner look, proper standard.
- **Cons:** Extra table and extra complexity for joins.

Rationale

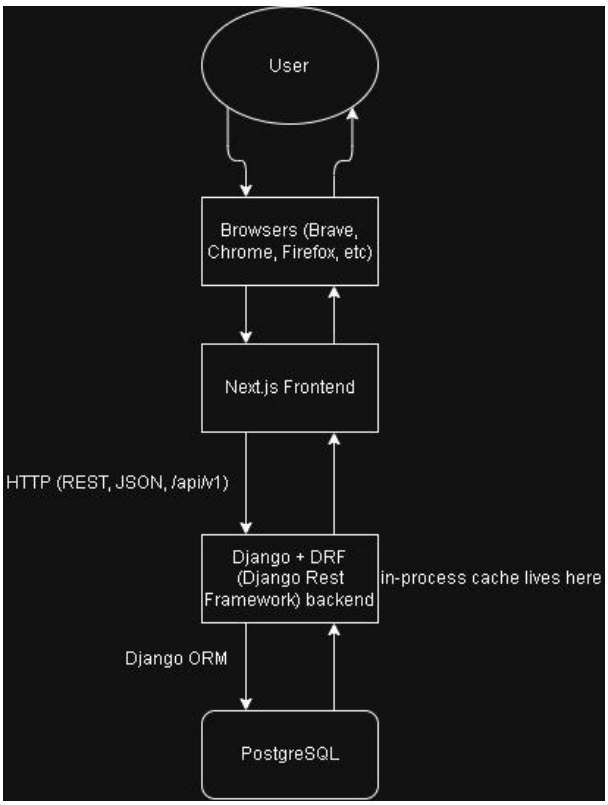
Option A (fixed columns) is the simpler and more appropriate choice for v1's stated requirement of one follow-up per record. However, if a future version needs to support multiple follow-ups per record, Option B avoids a breaking migration: the table already models the one-to-many shape, so relaxing `ref: -` to `ref: >` requires no data restructuring. Option B is chosen as a deliberate trade: one extra join today, to avoid a schema rewrite later.

Consequences

This choice adds a join on every GET /records/{id} and an extra table to include in cascade-delete rules, compared to Option A. In exchange, if a future version needs multiple follow-ups per record, the change is a one-line relationship update (`ref: -` → `ref: >`) rather than a breaking schema migration. Revisit this decision if a future requirement needs a follow-up history rather than a single reminder per record.

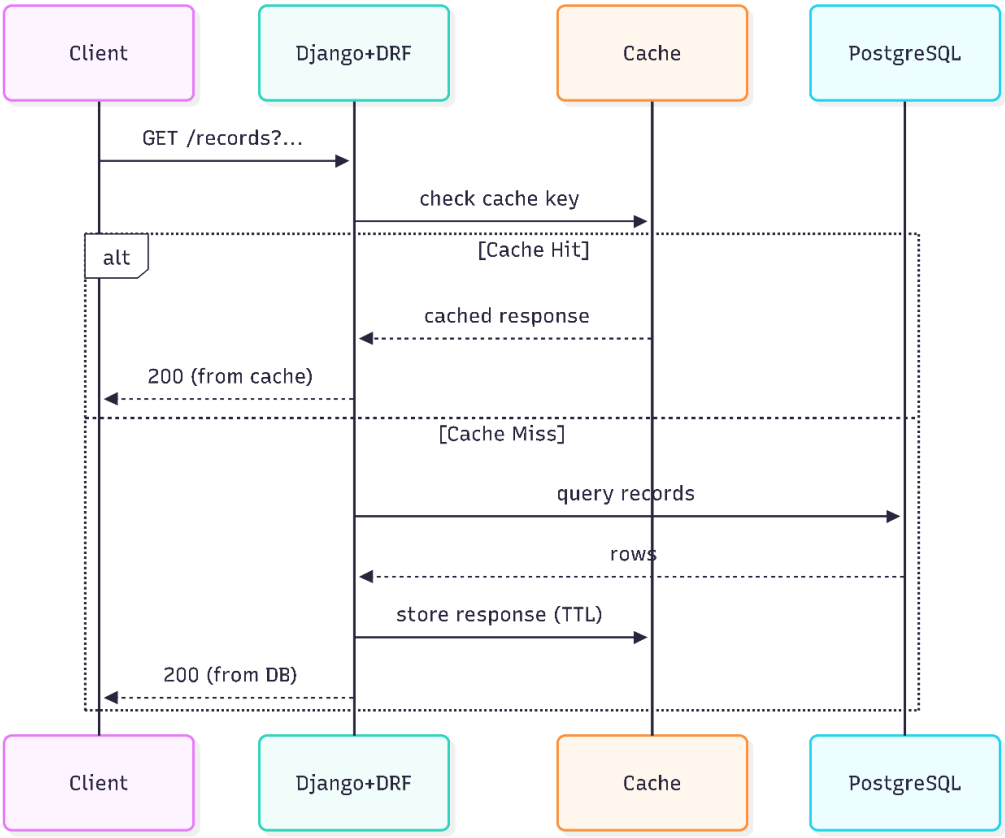
6. High-Level Design

6.1 Architecture diagram

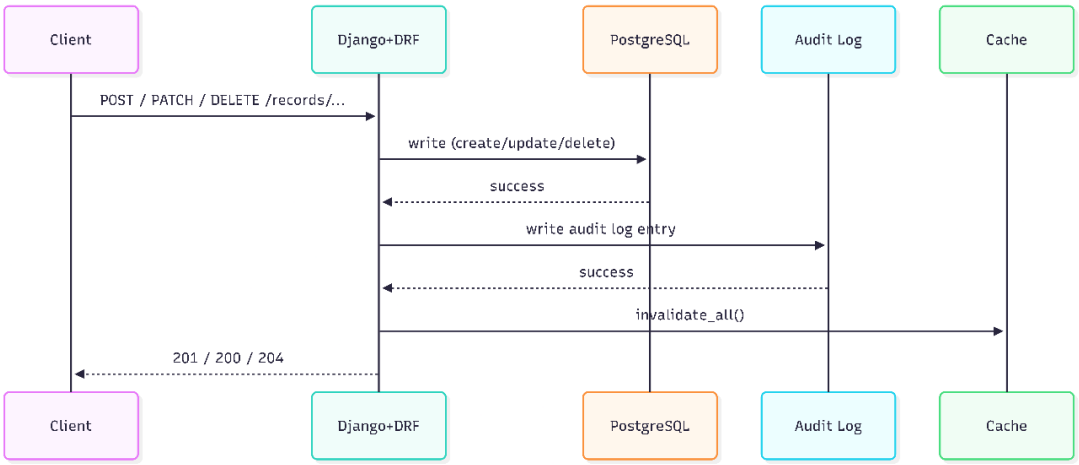


PostgreSQL and the Django backend run together via a single `docker-compose.yml` (two services, shared network) for local development, for environment isolation and easy teardown/rebuild. The Next.js frontend runs natively (`npm run dev`); containerization offers no benefit for the frontend dev workflow and can slow hot-reload.

Read (Cache Hit / Miss) Flow



Write (DB Write → Audit Log → Cache Eviction) Flow



Record writes and audit log inserts are executed within an atomic database transaction to guarantee no audit drift if an error occurs.

6.2 Decision: stack rationale, if any alternative was seriously considered

ADR-0002: PostgreSQL over SQLite / MySQL-MariaDB

Status: Accepted

Context

The options are PostgreSQL, SQLite, and MySQL/MariaDB. These are the options mainly because of developer familiarity and because they are the most common relational databases. A relational database was picked mainly because the project is a simple CRUD app using structured models. NoSQL was not considered, since the data is structured and relational by nature (records, notes, follow_ups, audit_logs with clear relationships).

Decision

PostgreSQL.

Alternatives considered

Option A: SQLite

- **Pros:** Zero configuration, single file, no separate server process to run or manage. Fully sufficient for this project's data volume (~0.95 MB at 6-month scale, per section 4). Simplest possible local development setup.
- **Cons:** Not commonly used in production/team environments or listed as a target skill for the roles this project is meant to build experience toward. Weaker support for concurrent writes if the project ever grows past single-user (v2 consideration, not a v1 blocker).

Option B: MySQL / MariaDB

- **Pros:** Also widely used in industry, well documented, broad hosting support.
- **Cons:** Historically weaker native support for case-insensitive text matching compared to Postgres (citext), which this project's schema uses for email handling (section 5). Would require a workaround such as storing emails lowercased. Note that the status/source enums are not a differentiator: Django stores them as text columns with a CHECK constraint on any of these databases (see section 7.1).

Option C: PostgreSQL

- **Pros:** Native citext for case-insensitive email handling, already used in the schema design (enabled through a Django migration). Strong standards compliance and widely considered a modern default for new backend projects. Directly relevant to the developer's target roles (e.g., DOST postings).
- **Cons:** Requires a running database server even in local development (via Docker or a local install), unlike SQLite's zero-setup, file-based approach.

Rationale

PostgreSQL was chosen primarily to align with the project's goal of building experience with a database commonly required in target roles (e.g., DOST postings). This is a deliberate trade: SQLite would technically be sufficient, and arguably simpler, for this project's actual scale (single user, < 1 MB of data per section 4). Postgres is chosen anyway because the skill-building goal outweighs the simplicity SQLite would offer at this scale.

This decision has a smaller technical benefit as well, beyond career goals: Postgres supports case-insensitive email handling natively through citext, which MySQL/MariaDB would need a workaround for. The status/source enums do not favor any of the three databases, since Django enforces them the same way on each (section 7.1).

Consequences

This decision would be revisited if requirements change to include media or large files, in which case Supabase would be used for both storage and database.

ADR-0003: Django + DRF over FastAPI

Status: Accepted

Context

The original plan was built around FastAPI with Pydantic schemas and SQLAlchemy/Alembic for the data layer. This was the first choice mainly because of familiarity and the simplicity of the project. It was then revisited for the same reason as ADR-0002: the developer is building experience with tools commonly required in target roles, and Django is requested more frequently than FastAPI. The decision was made after the API contract and data model had already been designed, and the alternatives were evaluated against the existing design (Alembic vs. Django migrations).

Decision

Django + DRF.

Alternatives considered

Option A: FastAPI (original plan)

- **Pros:** Native async support, minimal boilerplate for a pure API, automatic OpenAPI/Swagger docs generation, Pydantic validation built directly into route signatures.
- **Cons:** Less commonly listed as a target skill for the roles this project is meant to build experience toward. No built-in admin interface, ORM, or auth system; those would need to be separately chosen and wired in if the project grows.

Option B: Plain Django (no DRF)

- **Pros:** One framework to learn instead of two layered together; full visibility into the request/response cycle without a library abstracting it.
- **Cons:** No built-in request validation, pagination, or structured error formatting, all of which section 3.1/3.2 already assumes (pagination envelope, PATCH semantics, structured 400 error responses). Would require hand-building functionality that already exists in a standard library, for no added learning value specific to this project's stated goals.

Option C: Django + DRF

- **Pros:** Serializers (validation + response shaping) map directly onto the role Pydantic schemas were playing in the original design. Built-in pagination and generic views closely match the contract already designed in section 3. DRF is the de facto standard for building REST APIs in Django, so it's the realistic skill pairing for Django-based roles.
- **Cons:** Adds a second library's conventions (Serializer, ViewSet, generics classes) on top of Django itself, which is more to learn than Option A or B individually. More setup/boilerplate (settings, app structure, routers) than FastAPI's minimalism, for a project at this scale.

Rationale

Django + DRF was chosen primarily for the same career-relevance reason as PostgreSQL (ADR-0002): it is the stack most consistent with target roles. This is reinforced by a concrete technical fit: the API contract in section 3.1/3.2 (pagination envelope, PATCH-based partial updates, structured validation errors) was already designed in a shape that matches DRF's own built-in defaults, before DRF was chosen. This suggests the contract and the tool are well matched, rather than the tool being forced onto an unrelated design.

Consequences

This choice gives up FastAPI's native async support and lighter footprint, trading them for Django's batteries-included ecosystem (admin panel, auth system, ORM, migrations) and closer alignment with target job postings. It also means learning DRF's conventions in addition to Django's own, which

FastAPI + Pydantic would not have required as a second layer. Revisit this decision if a future version needs high-throughput async I/O or very low per-request latency at scale, conditions under which FastAPI's async-native design would offer a real advantage that Django's (largely synchronous, by default) request handling does not.

ADR-0004: Service layer for write side effects

Status: Accepted

Context

Every write to a record, note, or follow-up must also produce an audit log entry (FR-7.1) and clear the cache (NFR-9), and the first two must succeed or fail together in one atomic transaction (section 6.1). The question is where that logic lives. The options were the model layer, Django signals, or an explicit service layer called by the views.

Decision

Service layer: plain functions (e.g., create_record, update_record, delete_record, and the note and follow-up equivalents) that open the transaction, perform the write, write the audit entry, and clear the cache after commit. Views only validate input via serializers and call these functions.

Alternatives considered

Option A: Override save() / delete() on models

- **Pros:** Covers every write path automatically, including admin and shell.
- **Cons:** Skipped by bulk operations (QuerySet.update, bulk_create); the model has no access to the request or initiator; puts cache logic inside the data layer.

Option B: Django signals

- **Pros:** Keeps models clean; good for cross-app side effects.
- **Cons:** Cause and effect live in different files, so the flow is implicit and harder to trace; works against the goal of learning the cache invalidation sequence by seeing it.

Option C: Service layer

- **Pros:** The order (write, audit, invalidate) is explicit in one place; easy to unit test; the CSV import can reuse the same functions.
- **Cons:** Any write that bypasses the service functions is silently unaudited and leaves the cache stale.

Rationale

v1 is single user with only the API and the optional CSV import as write paths, so the bypass risk is small and controllable. The explicit sequence also serves the stated learning goal (section 1.2).

Consequences

The Django admin must not be used to edit these models, and no code may call QuerySet.update() or bulk_create() on them directly; the CSV import must go through the service functions. Cache invalidation should run after the transaction commits so a rollback never clears the cache for a write that did not happen. Revisit if v2 adds more write paths (background jobs, multiple apps) or real users, where a model-level or signal-based safety net may be worth adding.

7. Low-Level Design

7.1 Django models

The four tables in section 5.1 map to four Django models: Record, Note, FollowUp, and AuditLog. Writes to them go through the service layer (ADR-0004), not through the models.

- **Primary keys:** Custom auto-increment primary keys (record_id, note_id, follow_up_id, log_id) so the schema and the API paths in section 3 match exactly.
- **Note to Record:** Foreign key, delete cascades (NFR-3).
- **FollowUp to Record:** One-to-one, delete cascades. Relaxing this to a foreign key is the planned v2 change from ADR-0001.
- **AuditLog:** No foreign key; entity_type and entity_id are plain values so entries survive deletion of the parent (NFR-3). Append-only: no update or delete path exists (NFR-5). old_value and new_value are strings, null on create and delete respectively.
- **Timestamps:** created_at and updated_at on records, notes, and follow_ups; audit logs carry created_at only (UTC).
- **Status and source:** Restricted to the values in section 5.1 (status: contacted, ongoing, completion, dropped). Enforced at two levels: Django choices (validation in Python, and the DRF serializer error) plus a database CHECK constraint, so invalid values are rejected even if a write bypasses the service layer. Stored as text columns, not native Postgres enum types, because adding a value to a native enum needs a more awkward migration and Django has no built-in support for it. Values are added by changing the choices and the constraint in one reversible migration (NFR-4).
- **Email:** company_email uses the citext type (case-insensitive text), so lookups and any future uniqueness rule ignore case without extra code. citext is a Postgres extension and must be enabled in a migration that runs before the migration creating the table; the migration must be reversible (NFR-4). Not required by any v1 requirement (no email search or uniqueness), chosen to practice a Postgres-specific feature in line with ADR-0002.
- **Indexes (planned, to be verified):** B-tree indexes on records.status (FR-3.1), records.last_date_contacted (FR-3.2), and follow_ups.follow_up_date (FR-4.1); trigram (pg_trgm) indexes on records.company and records.source for the partial, case-insensitive search (FR-3.3), since a B-tree cannot serve a match that can start anywhere in the text. pg_trgm is enabled by a reversible migration, like citext. A combined status + last_date_contacted index is deferred until a measurement shows it is needed.

Verification: at v1 scale (~540 to 1,800 records) the query planner will likely ignore these indexes and scan the table, so the comparison needs a larger seeded dataset (e.g., 100,000 rows) to show a difference. Record EXPLAIN ANALYZE output for the list and search queries (a) with only primary and foreign key indexes and (b) with the planned indexes, with the cache bypassed (NFR-10). Then confirm NFR-1 and NFR-2 at 1,000 realistic records. Keep an index only if the measurement justifies it; if it does not, record that finding here instead.

7.2 DRF Serializers

Serializers validate input and shape output; they do not write audit entries or touch the cache, since that belongs to the service layer (ADR-0004). The record is represented two ways because the technical audit text and notes are long, manually written content that the list view does not need.

- **RecordListSerializer (read-only):** Used for items in GET /records. Slim summary: record_id, company, status, source, last_date_contacted, follow_up_date, and a computed

follow_up_due flag (true when follow_up_date is today or earlier, FR-4.1). Excludes notes, technical_audit, and contact details.

- **RecordDetailSerializer (read-only):** Used for GET /records/{record_id}. All record fields, including contact details and technical_audit, plus its notes and its follow-up.
- **RecordWriteSerializer:** Used for POST and PATCH. Validates that company and status are present on create (FR-1.1), that status and source are within the allowed values, and that email and dates are well formed. PATCH is partial only (section 3.2). Returns the updated object in the detail shape.
- **NoteSerializer:** record_id (read-only, from the URL), content, created_at, updated_at. created_at and updated_at are server-set and read-only (FR-2.1).
- **AuditLogSerializer (read-only):** Every field is read-only; there is no write path (NFR-5).

Follow-up writes (nested object): the client sets a follow-up through PATCH /records/{record_id} with a follow_up object (follow_up_date, content), mirroring the one-to-one table. DRF does not save nested objects automatically, so the serializer only validates the nested data and the service layer performs the create, update, or delete explicitly.

- **follow_up key omitted:** follow-up is left unchanged.
- **follow_up object, no follow-up exists yet:** a follow-up row is created.
- **follow_up object, follow-up exists:** updated partially, only the keys sent.
- **follow_up set to null:** the follow-up row is deleted.

Each of these changes writes its own audit entry with entity_type follow_up (FR-7.1), in the same transaction as any change to the record itself, and clears the cache (NFR-9). The list view keeps a flat read-only follow_up_date for brevity, while the detail view and PATCH use the nested follow_up object. Trade-off: moving to multiple follow-ups per record (ADR-0001) would turn follow_up into a follow_ups list, which is a breaking API change and would need /api/v2.

POST /records does not accept a follow_up object. Follow-ups are added only through a later PATCH, which keeps creation to the fields in FR-1.1 and the create path simple.

7.3 Django migrations

Schema changes happen only through Django migrations (NFR-4). Planned migration sequence:

- **1. Extensions:** a migration that enables citext and pg_trgm (Django provides operations for both in django.contrib.postgres). It must run before any migration that uses them.
- **2. Initial models:** records, notes, follow_ups, and audit_logs, including the CHECK constraints on status and source (section 7.1).
- **3. Indexes:** a separate migration, so the planned indexes can be added or dropped independently while they are being measured (section 7.1).

Every migration must be reversible: after writing one, migrate forward, then back to the previous state, then forward again, and confirm the schema is unchanged. Adding a new status or source value is its own migration that updates the choices and replaces the CHECK constraint. Creating extensions needs database privileges; the default user in the local docker-compose Postgres container has them, and the official image ships both extensions.

7.4 Route handlers

Route handlers are DRF ViewSets registered with a router (the choice of ViewSets over explicit views follows the career-relevance goal, as in ADR-0003). Views validate input through serializers and call the service layer for every write (ADR-0004); they never call the serializer's save() directly.

- **RecordViewSet:** list, retrieve, create, partial update, and destroy, with PUT disabled (section 3.2). Custom actions: notes (POST, creates a note under a record), export (GET, CSV), and import (POST, CSV). The detail routes use record_id as the URL parameter. DRF registers the export and import list routes before the detail route, so they are not captured as a record ID.
- **NoteViewSet:** partial update and destroy only, at /notes/{note_id}. Creation goes through the record's notes action.
- **AuditLogViewSet:** list only (GET /audit-logs), read-only (NFR-5).
- **Write hooks:** the ViewSet's create, update, and destroy hooks are overridden to call the matching service function instead of the default save. PATCH returns the updated record in the detail shape (section 3.2).
- **List query:** get_queryset applies status, q (case-insensitive partial match on company or source), follow_up_due, sort_by, and order. sort_by is checked against an allowed list of fields. The follow-up is joined in one query (select_related), and notes are prefetched for the detail view, to avoid one query per record.
- **Pagination:** a custom page-number paginator. DRF's default response shape (count, next, previous, results) does not match the contract, and its default for an out-of-range page_size is to clamp it rather than return 400, so both behaviors are overridden to match section 3.1.
- **Caching:** the list action reads through the cache, keyed by the full set of query parameters, with a 30 s TTL (NFR-9). A hit returns the stored response body; a miss runs the query and stores it. The service layer clears the whole cache after a successful write (ADR-0004). LocMemCache is per-process, which is acceptable for v1 (single user, one server process) and is revisited in 8.1.
- **List filters (django-filter):** a FilterSet on the record list declares status, q, follow_up_due, sort_by, and order. status is an exact match limited to the allowed values. q and follow_up_due are custom filter methods: q matches company or source case-insensitively on partial text, and follow_up_due compares the joined follow-up date with today. sort_by is limited to an allowed list of fields and combined with order. django-filter's built-in ordering filter uses a single parameter, so sort_by and order are handled by a custom method to keep the contract in section 3.1. An invalid filter value returns 400, which matches the contract.
- **Audit log filters:** GET /audit-logs supports four filters: entity_type, entity_id, action, and a created_at range (created_after, created_before). entity_id is only meaningful together with entity_type, since a note and a record can share the same number. Results are newest first and use the same pagination as the records list.

8. Bottlenecks & Future Considerations

CDN and reverse-proxy caching (e.g., Nginx/Varnish in front of Django) is not needed at v1 scale (single user, concurrency = 1, see section 4). Revisit if the app moves to multi-user with geographically distributed traffic, or if static asset volume grows enough that Next.js's own deployment-level CDN (e.g., via Vercel) isn't sufficient on its own.

8.1 What breaks at multi-user scale

These are the v1 decisions that stop working once there is more than one user or more than one server process:

- **In-process cache:** LocMemCache is private to each server process. With several workers, a write clears only the cache of the worker that handled it, so the others can serve stale lists for up to the 30 s TTL. A shared cache (Redis, deferred to v2 in section 1.2) fixes this.

- **Full-cache invalidation:** clearing the whole cache on any write is harmless for one user, but with many users writing, the cache would be cleared so often that it rarely hits. Fine-grained invalidation (a v1 non-goal) would be needed.
- **Concurrent edits:** PATCH is last-write-wins, so two users editing the same record can silently overwrite each other. Fixing it needs optimistic locking, for example rejecting an update when the record's updated_at no longer matches what the client last read.
- **No ownership or auth:** records have no owner, and every endpoint is open (NFR-6). Multi-user needs authentication, an owner on each record, and every query filtered by owner. Sequential integer IDs would let one user guess another's record URLs unless that filtering is enforced.
- **Audit initiator:** audit_logs.initiator is the placeholder "system" (section 5.1) and must come from the authenticated user.

8.2 v2 candidates (auth, concurrent writes, data isolation)

No v2 scope is committed yet. The one planned candidate is authentication, motivated by deploying the app publicly (for example, the Next.js frontend on Vercel).

- **Authentication (FR-6.1 moves from Won't to Must):** login and accounts, an owner on every record, and every query and write filtered by that owner (data isolation). The audit log initiator is filled from the authenticated user. How the Next.js frontend authenticates against the Django API (session cookies or tokens) is undecided and needs its own ADR.

Deploying publicly also invalidates several v1 assumptions beyond auth itself:

- **Security posture:** NFR-6 (localhost, no auth) no longer holds. The API needs HTTPS, production Django settings (debug off, allowed hosts set, secrets from the environment), and CORS pointed at the deployed frontend origin instead of localhost:3000.
- **Hosting split:** the frontend and the Django API plus Postgres are separate deployments with different needs. Vercel is built mainly for the Next.js side; the API needs a host that can run Django and a managed Postgres, and in-process caching assumptions change on any host that runs several workers or short-lived instances (section 8.1). Hosting options should be checked against current provider documentation before choosing.
- **Shared cache and locking:** move from LocMemCache to a shared cache, and add optimistic locking for concurrent edits (section 8.1).

Other candidates, not committed: multiple follow-ups per record (ADR-0001, a breaking API change under /api/v2) and telemetry (FR-8.1).

If this project moves toward team development or production deployment, revisit containerizing the frontend as part of a full docker-compose stack. Local dev and deployment have different needs, and this decision applies to local dev only.

9. Testing Plan

9.1 What Pytest covers for v1

Tests run with pytest against the same Postgres used in development (the docker-compose service, using a separate test database), not SQLite, because the migrations depend on citext and pg_trgm. Coverage is measured on the Django views and viewsets and must be at least 80% (NFR-7); ruff runs alongside the tests in CI (NFR-8).

- **API contract:** every endpoint returns the documented success and error status codes, the pagination envelope, and a 400 for an invalid page, page_size, filter value, or follow-up/status/source value (sections 3.1 and 3.2).

- **Write flow:** a successful create, update, or delete writes exactly one audit entry per change and clears the cache; a write that fails midway leaves no record change and no audit row (section 6.1, ADR-0004).
- **Cache:** a repeated list request is a hit; a different query is a miss; an entry expires after the 30 s TTL (using a controlled clock, not a real wait); any record, note, or follow-up write invalidates it (NFR-9).
- **Data integrity:** deleting a record removes its notes and follow-up and leaves no orphans, while its audit entries remain (NFR-3). Audit logs have no update or delete endpoint (NFR-5).
- **Migrations:** each migration applies forward, reverses, and applies again with the schema unchanged (NFR-4).
- **Performance:** with 1,000 seeded records and the cache bypassed, the list and search endpoints stay under 500 ms at p95 (NFR-1, NFR-2, NFR-10). Timing tests can be flaky on shared CI machines, so this check may be run manually or with a generous margin if it proves unreliable.